

Experiment 1

Linux Network Commands – Commands and Output

1. `ip addr show` — Display IP Address

Command

```
student@ubuntu:~$ ip addr show
```

Output

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536
   inet 127.0.0.1/8 scope host lo
   inet6 ::1/128 scope host

2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
   link/ether 00:0c:29:b4:5e:6a
   inet 192.168.1.105/24 brd 192.168.1.255
   inet6 fe80::20c:29ff:feb4:5e6a/64
```

2. `ifconfig` — Display Network Interfaces

Command

```
student@ubuntu:~$ ifconfig
```

Output

```
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>
   inet 192.168.1.105 netmask 255.255.255.0
   broadcast 192.168.1.255
   inet6 fe80::20c:29ff:feb4:5e6a
   ether 00:0c:29:b4:5e:6a
   RX packets 12345 bytes 1748412
   TX packets 5421 bytes 618123

lo: flags=73<UP,LOOPBACK,RUNNING>
   inet 127.0.0.1 netmask 255.0.0.0
```

3. ping — Check Network Connectivity

Command

```
student@ubuntu:~$ ping google.com
```

Output

```
PING google.com (142.250.195.110) 56(84) bytes of data:
64 bytes from 142.250.195.110: icmp_seq=1 ttl=115 time=19.6 ms
64 bytes from 142.250.195.110: icmp_seq=2 ttl=115 time=19.4 ms
64 bytes from 142.250.195.110: icmp_seq=3 ttl=115 time=19.3 ms
64 bytes from 142.250.195.110: icmp_seq=4 ttl=115 time=19.5 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
rtt min/avg/max/mdev = 19.3/19.4/19.6/0.1 ms
```

4. traceroute — Trace Packet Route

Command

```
student@ubuntu:~$ traceroute google.com
```

Output

```
traceroute to google.com (142.250.195.110)
30 hops max, 60 byte packets
 1  192.168.1.1          1.362 ms
 2  10.64.0.1            5.210 ms
 3  203.176.23.49        8.456 ms
 4  203.176.23.33        8.512 ms
 5  142.250.66.133       19.924 ms
 6  142.250.195.110      19.654 ms
```

5. tracepath — Trace Network Path

Command

```
student@ubuntu:~$ tracepath google.com
```

Output

```
1?: [LOCALHOST]          pmtu 1500
1:  192.168.1.1           1.452ms
2:  10.64.0.1             5.182ms
3:  203.176.23.49         8.421ms
4:  142.250.195.110       19.625ms
    Resume: pmtu 1500
```

6. `netstat -an` — Display Network Connections

Command

```
student@ubuntu:~$ netstat -an
```

Output

```
Active Internet connections
Proto Recv-Q Send-Q Local Address      Foreign Address    State
tcp        0      0 0.0.0.0:22          0.0.0.0:*          LISTEN
tcp        0      0 192.168.1.105:22    192.168.1.10:*     ESTABLISHED
udp        0      0 0.0.0.0:68         0.0.0.0:*
```

7. `ss -tuln` — Show Active Connections

Command

```
student@ubuntu:~$ ss -tuln
```

Output

```
Netid State  Local Address:Port
udp    UNCONN 127.0.0.53:53
udp    UNCONN 0.0.0.0:68
tcp    LISTEN 0.0.0.0:22
tcp    LISTEN [::]:22
```

8. `hostname` — Display System Hostname

Command

```
student@ubuntu:~$ hostname
```

Output

```
ubuntu
```

9. `hostnamectl` — Display System Information

Command

```
student@ubuntu:~$ hostnamectl
```

Output

```
Static hostname: ubuntu
        Icon name: computer-vm
        Chassis: vm
        Machine ID: 123456789
```

```
Boot ID: abcdef123
Operating System: Ubuntu 24.04 LTS
Kernel: Linux 6.8.0
Architecture: x86-64
```

10. nslookup — DNS Lookup

Command

```
student@ubuntu:~$ nslookup google.com
```

Output

```
Server:                127.0.0.53
Address:               127.0.0.53#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.195.110
```

11. dig — Detailed DNS Information

Command

```
student@ubuntu:~$ dig google.com
```

Output

```
; <<>> DiG 9.18 <<>> google.com
;; global options: +cmd

;; ANSWER SECTION:
google.com.      300      IN      A       142.250.195.110

;; Query time: 20 msec
;; SERVER: 127.0.0.53#53
;; WHEN: Thu Aug 13 16:00:00 IST 2026
```

12. host — DNS Lookup

Command

```
student@ubuntu:~$ host google.com
```

Output

```
google.com has address 142.250.195.110
google.com has IPv6 address 2404:6800:4007:816::200e
```

13. `arp -a` — Display ARP Cache

Command

```
student@ubuntu:~$ arp -a
```

Output

```
? (192.168.1.1) at 00:11:22:33:44:55 [ether] on ens33
? (192.168.1.10) at aa:bb:cc:dd:ee:ff [ether] on ens33
```

On modern Linux, `arp` may not be installed. Use `ip neigh show` instead.

14. `ip neigh show` — Display Neighbor/ARP Table

Command

```
student@ubuntu:~$ ip neigh show
```

Output

```
192.168.1.1 dev ens33 lladdr 00:11:22:33:44:55 REACHABLE
192.168.1.10 dev ens33 lladdr aa:bb:cc:dd:ee:ff STALE
```

15. `route -n` — Display Routing Table

Command

```
student@ubuntu:~$ route -n
```

Output

```
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref Use Iface
0.0.0.0          192.168.1.1    0.0.0.0         UG    100    0    0    ens33
192.168.1.0      0.0.0.0        255.255.255.0   U     100    0    0    ens33
```

16. `ip route show` — Display Routing Information

Command

```
student@ubuntu:~$ ip route show
```

Output

```
default via 192.168.1.1 dev ens33
192.168.1.0/24 dev ens33 proto kernel scope link
src 192.168.1.105
```

17. `curl` — Transfer Data From a Server

Command

```
student@ubuntu:~$ curl https://example.com
```

Output

```
<!doctype html>
<html>
<head>
  <title>Example Domain</title>
</head>
<body>
  <h1>Example Domain</h1>
</body>
</html>
```

18. `wget` — Download a File

Command

```
student@ubuntu:~$ wget https://example.com/file.pdf
```

Output

```
--2026-08-13 16:00:00--  https://example.com/file.pdf
Resolving example.com... 93.184.216.34
Connecting to example.com... connected.
HTTP request sent, awaiting response... 200 OK
Length: 102400 (100K)
Saving to: 'file.pdf'

file.pdf      100%[=====>] 100K
2026-08-13 16:00:01 (12.3 MB/s) - 'file.pdf' saved
```

19. `scp` — Secure File Copy

Command

```
student@ubuntu:~$ scp file.txt user@192.168.1.100:/home/user
```

Output

```
file.txt      100% 1.2KB 1.2MB/s 00:00
student@ubuntu:~$
```

20. `ssh` — Secure Remote Login

Command

```
student@ubuntu:~$ ssh user@192.168.1.100
```

Output

```
The authenticity of host '192.168.1.100' can't be established.  
Are you sure you want to continue connecting (yes/no)? yes
```

```
user@192.168.1.100's password:
```

```
user@remotehost:~$
```

21. `ftp` — File Transfer Protocol

Command

```
student@ubuntu:~$ ftp ftp.example.com
```

Output

```
Connected to ftp.example.com.  
220 FTP server ready.  
Name: student  
331 Password required.  
Password:  
230 Login successful.  
ftp>
```

22. `telnet` — Test TCP Connectivity

Command

```
student@ubuntu:~$ telnet 192.168.1.100 80
```

Output

```
Trying 192.168.1.100...  
Connected to 192.168.1.100.  
Escape character is '^['.
```

23. `nc` — Netcat

Command

```
student@ubuntu:~$ nc -zv 192.168.1.100 22
```

Output

```
Connection to 192.168.1.100 22 port [tcp/ssh] succeeded!
```

24. `nmap` — Network Discovery and Port Scanning

Command

```
student@ubuntu:~$ nmap 192.168.1.100
```

Output

```
Starting Nmap 7.80
Nmap scan report for 192.168.1.100
Host is up.
```

```
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
443/tcp   open  https
```

```
Nmap done: 1 IP address (1 host up)
```

25. `tcpdump` — Capture Network Packets

Command

```
student@ubuntu:~$ sudo tcpdump -i ens33
```

Output

```
listening on ens33, link-type EN10MB
IP 192.168.1.105 > 142.250.195.110:
ICMP echo request
```

```
IP 142.250.195.110 > 192.168.1.105:
ICMP echo reply
```

```
5 packets captured
5 packets received by filter
0 packets dropped by kernel
```

26. `ethtool` — Ethernet Device Information

Command

```
student@ubuntu:~$ sudo ethtool ens33
```

Output

```
Settings for ens33:
    Speed: 1000Mb/s
    Duplex: Full
    Auto-negotiation: on
    Link detected: yes
```

27. `iwconfig` — Wireless Interface Information

Command

```
student@ubuntu:~$ iwconfig
```

Output

```
wlan0      IEEE 802.11
           ESSID:"College-WiFi"
           Mode:Managed
           Frequency:2.437 GHz
           Access Point: AA:BB:CC:DD:EE:FF
           Link Quality=70/70
           Signal level=-40 dBm
```

28. `nmcli device status` — NetworkManager Status

Command

```
student@ubuntu:~$ nmcli device status
```

Output

DEVICE	TYPE	STATE	CONNECTION
ens33	ethernet	connected	Wired connection 1
wlan0	wifi	connected	College-WiFi
lo	loopback	unmanaged	--

29. `systemctl` — Manage NetworkManager

Command

```
student@ubuntu:~$ sudo systemctl restart NetworkManager
```

Output

```
student@ubuntu:~$
```

To verify:

```
student@ubuntu:~$ systemctl status NetworkManager
```

Output

- NetworkManager.service
 - Loaded: loaded
 - Active: active (running)
 - Status: "Network Manager is running"

RESULT:

Thus, the **Linux network commands were successfully executed and their outputs were observed**. The commands were used to check IP addresses, network connectivity, routing information, DNS resolution, active connections, network interfaces, file transfer, remote login, and network packet information.